

Turn 1

Assistant

这是一份严格按照您上传的课件内容整理的 Virtual Reality 复习笔记。笔记已按照三级结构（章节、小节、知识点）进行了系统化重构，对核心渲染管线、公式、软硬件瓶颈等内容进行了重点梳理。

一、VR System Architecture (VR系统架构)

1.1 VR 系统的核心任务与架构要求

- 核心任务：
 - 读取输入设备（来自传感器的数据）。
 - 更新虚拟世界的状态
 - 渲染输出内容（如图形、触觉等）。
 - 将输出内容传送给输出设备
- **VR 引擎架构要求：** VR 引擎必须具备强大的计算机架构，以提供极低的计算延迟（Low latency）、快速的图形渲染（Fast graphics rendering）以及触觉渲染（Haptics rendering）。

二、Graphics Rendering Pipeline (图形渲染管线)

2.1 渲染管线概述与 Application Stage (应用阶段)

- 渲染的定义：将虚拟世界中的 3D 几何模型转换为 2D 图像呈现给用户的过程（此概念也可扩展至触觉渲染）。
- 管线化（**Pipelining**）：指将渲染过程分为多个阶段（Stages），并将这些阶段分配给不同的硬件资源

- 管线三大阶段：依次为 Application stage（应用阶段）、Geometry stage（几何阶段）、Rasterizer stage（光栅化阶段）。
- **Application Stage（应用阶段）** workflow:
 - 执行环境：纯软件执行，由 CPU 负责
 - 核心功能：管理虚拟世界的几何数据库
 - 执行循环：
 1. 获取用户输入（如鼠标、追踪器、传感手套）
 2. 更新视图、数据库以及虚拟世界的状态
 3. 选择相关的几何对象（Geometry objects），并将其发送到后续阶段进行渲染
 - 【图示文字化说明：管线架构图】 用户输入进入 Application Stage (CPU内的多个处理器) → 存入 FIFO 缓冲 → 进入 Geometry Stage (GPU内的多个几何引擎) → 存入 FIFO 缓冲 → 进入 Rasterizer Stage (GPU内的多个光栅化单元) → 输出至输出缓冲区与视频控制器

2.2 Geometry Stage（几何阶段）

- 执行环境：通常由硬件（GPU）实现
- 输入内容：接收来自应用阶段的几何图元（Geometry primitives）列表
- 核心任务：模型变换（Model transformations）、着色计算（Shading computations）、场景投影（Scene projection）、裁剪（Clipping）。
- **Illumination（光照模型）**：
 - 原理：描述在模拟物体表面某像素上反射并被观察者看到的光强度 表面颜色计算基于：模拟光源的类型与数量、光照模型、物体的反射/透射属性、以及雾/烟等大气效应
 - **Phong Illumination Model（Phong 光照模型公式）**：
$$I_{\lambda} = I_{a\lambda}K_aC_{d\lambda} + f_{att}I_{p\lambda}(K_dC_{d\lambda}\cos\theta + K_sC_{s\lambda}\cos^n\alpha)$$
 - 变量含义与组成部分：
 - 第一部分为 Ambient reflection（环境反射），第二部分的加和中包含 Diffuse reflection（漫反射）与 Specular reflection（镜面反射）。
 - I_{λ} ：波长为 λ 的光强度

- $I_{a\lambda}$: 环境光强度
- K_a, K_d, K_s : 分别为表面的环境反射系数、漫反射系数、镜面反射系数
- $C_{d\lambda}, C_{s\lambda}$: 分别为物体的漫反射颜色和镜面反射颜色
- f_{att} : 大气衰减因子
- $I_{p\lambda}$: 波长为 λ 的点光源强度
- θ, α : 分别是光线向量与法向量的夹角、以及反射光与观察视角的夹角
- 多光源扩展: 当引入多个光源时, 光源项需进行累加计算 由于 I_λ 的计算成本随光源数量增加而增加, 这可能成为性能瓶颈, 因此某些 GPU 会限制支持的最大光源数
- 镜面反射特性: 模拟汽车外观或镜子等发光表面的反射光; 当反射光线 R 与观察方向 V 之间的角度 α 较小时, 反射光最强
- **The Polygon Shading Model (多边形着色模型对比)** :
 - **Flat Shading (平直着色)** : 为对象多边形内的所有像素分配相同的颜色
 - **Gouraud Shading (高洛德着色)** : 根据顶点的颜色, 在多边形内部对颜色进行插值计算
 - **Phong Shading (冯氏着色)** : 在计算每个像素的光照强度之前, 先在多边形内部对顶点法线进行插值, 这是最逼真的着色模型

2.3 Rasterizer Stage (光栅化阶段)

- 执行环境: 通常由硬件实现
- 核心功能: 将来自几何阶段的顶点信息 (x, y, z, 颜色, 纹理) 转换为用于显示的像素信息
- 核心任务:
 - 将每个几何图元 (通常为微小的三角形或矩形) 分解为像素片段 (Pixel fragments)
 - 抗锯齿 (Antialiasing)
 - 纹理映射 (Texture mapping)
- 抗锯齿 (Antialiasing) 工作流与原理:
 - 将每个像素细分为 n 个子像素 (Sub-samples)
 - 计算每个子像素的颜色

- 最终像素颜色为所有子像素颜色的平均值（例如，16次采样中，5个落在蓝色区域，11个落在绿色区域，最终颜色为 $5/16$ 蓝 + $11/16$ 绿）。
- 通常，每个像素的子采样越多，抗锯齿效果越好（如 16 sub-samples 优于 8 sub-samples）。
- 问题：增加每个像素的子采样数会导致计算量大幅增加
- 缓冲区（**Buffers**）机制：
 - 像素颜色信息存储在 Color buffer（颜色缓冲区）中
 - 像素的 z 值存储在 Z-buffer（深度缓冲区）中，用于确定对象的可见性（仅可见对象的颜色存入颜色缓冲区）。
 - **Double Buffering**（双重缓冲）：渲染时通常使用两个颜色缓冲区，Front buffer（前台缓冲）存储当前正在屏幕上显示的最新渲染图像；Back buffer（后台缓冲）存储当前正在渲染的图像
 - 渲染完成后，前后缓冲区交换（Buffer swapping），新图像显示于屏幕 此过程可有效消除画面闪烁（Flicker）。
- 纹理映射（**Texture Mapping**）：纹理信息指示从纹理图像中的哪个位置获取像素颜色，将 2D 纹理贴在 3D 模型表面

2.4 Graphics Pipeline Bottlenecks（管线瓶颈分析）

- 理想情况：某一阶段的输出速率与下一阶段的输入速率完美匹配
- 实际情况瓶颈诊断（三种核心限制）：
 - **"CPU-limited"**（应用阶段瓶颈）：如果 CPU 运行率达到 100%
 - **"Transform-limited"**（几何阶段瓶颈）：如果减少光源数量或多边形数量能提升性能
 - **"Fill-limited"**（光栅化阶段瓶颈）：如果降低图像分辨率或多边形数量能提升性能
- 【图示文字化说明：帧率曲线图】真实帧率（Real frame rate）在场景复杂度（多边形数量）增加时会急剧下降，由于 CPU 计算能力和数据带宽有限，应用阶段能提供的最高帧率存在上限（例如 Stereo mode 与 Mono mode 在低复杂度时均有一条水平截断的最高帧率线）。

2.5 Graphics Pipeline Optimization（管线优化策略）

- 应用阶段优化 (**Application Stage**) :
 - 硬件升级: 更换更快的 CPU (高频率或多核)、使用更快的 RAM
 - 软件层: 使用更好的编译器
 - 编程技巧: 降低计算精度 (少用双精度)、尽量减少除法运算
- 几何阶段优化 (**Geometry Stage**) :
 - 减少光源数量
 - 降低场景复杂度: 减少几何模型中的多边形数量 (使用包含数百个多边形的低分辨率模型替代包含数千个多边形的高分辨率模型)
- 光栅化阶段优化 (**Rasterizing Stage**) :
 - 降低图像分辨率 (即减少需要处理的像素总数)
 - 与几何阶段相同, 减少多边形数量

三、 The Haptics Rendering Pipeline (触觉渲染管线)

3.1 触觉管线三大阶段

- **Collision Detection Stage** (碰撞检测阶段) :
 - 加载物体的物理特征 (如表面光滑度、表面温度和重量)
 - 执行碰撞检测以判断用户与物体之间是否发生碰撞
 - 将发生碰撞的结构传递给下一阶段
- **Force Computation Stage** (力计算阶段) :
 - 计算力 (**Compute force**) : 基于物理模拟模型计算碰撞力; 碰撞物体越多、计算越复杂, 此阶段成为瓶颈 (Force-limited) 的可能性越高
 - 力平滑 (**Force smoothing**) : 调整力向量的方向, 以避免急剧的过渡变化
 - 力映射 (**Force mapping**) : 将计算出的力投影/映射到特定触觉设备的特性上
- **Tactile Computation Stage / Haptic Texturing** (触觉计算/触觉纹理阶段) :
 - 渲染模拟的触摸反馈部分
 - 将计算出的效果 (如振动或表面温度) 添加到力向量中, 并发送到触觉显示器

3.2 渲染管线解耦与同步 (Graphics and Haptics Pipeline Synchronization)

- 执行位置： 管线同步是在应用阶段 (Application stage) 完成的
- 力计算位置的两种实现： 1) 在主机计算机上进行； 2) 由嵌入在触觉接口控制器中的处理器进行
- 解耦的必要性 (**Decoupling necessity**)： 必须将图形管线和触觉管线解耦，因为它们的输出速率存在显著差异 例如，PHANTOM 触觉控制器的渲染频率高达 1KHz，而图形管线的渲染频率仅为 30-60 帧/秒 (FPS)。

四、 GPU Architecture (GPU 架构)

4.1 CPU 与 GPU 特性对比

- 发展速度： GPU 的增长速度 (十年约 1000 倍，每年约 2.0x) 远超 CPU (十年约 60 倍，每年约 1.5x)。例如 RTX 5090 拥有 21,760 个 CUDA 核心，32GB 内存，功耗 575W
- CPU 特性总结：
 - 使用标量编程模型，无原生数据并行性
 - 擅长控制密集型任务 (Control-heavy tasks)。
 - 不擅长数据密集型应用 (Data-heavy applications)。
 - 算术单元少 (空间不足)。
 - 针对低延迟 (Low latency) 进行优化，不支持高带宽
- 图形渲染/GPU 任务特性总结：
 - 需要处理海量数据与庞大计算量
 - 高度并行化 (Lots of parallelism)，控制逻辑简单
 - 多阶段的前馈管线 (Feed forward pipelines)。
 - 对延迟具有容忍度 / 管线深度大 (Latency-tolerant / deep pipelines)。
- 芯片空间分配差异： 相比 CPU 划分大量空间给控制 (Control) 和缓存 (Cache)，GPU 将绝大部分芯片空间用于计算单元 (ALU)，以适应其并行的海量运算需求

4.2 现代 GPU 硬件管线演进

- 高度可编程性：现代 GPU 具备可编程的顶点核心（Vertex cores）和片段核心（Fragment cores），支持高级语言编程
- 精度要求：整个渲染管线支持 32 位浮点计算（单精度），足以满足许多应用的需求
- **Vertex Processors**（顶点处理器 - 多个并行运行）：
 - 主要任务包括：变换（Transformation）、逐顶点光照计算（Per-vertex lighting computation）、背面剔除（Back-face removal）、裁剪（Clipping）。
- **Fragment Processors**（片段处理器 - 多个并行运行）：
 - 在此之前的光栅化（Rasterizer）负责将几何顶点数据转换为片段数据（片段即包含颜色、深度的像素数据），并对像素进行逐顶点变量的插值
 - 片段处理器任务：深度比较（Depth comparison）、计算每个像素的颜色、从纹理读取颜色选项
- **GPU 内存寻址**：当前 GPU 不支持直接内存寻址，仅支持向量寻址（如 1D, 2D, 3D），这种简单的寻址方案允许数据在管线中进行高效的大规模传输

五、 Distributed VR Architecture（分布式 VR 架构）

5.1 架构分类与显示布局

- 定义：使用两个或更多渲染管线的 VR 引擎 这些管线可位于单台计算机、同地点的多台计算机或远程的分布式网络中
- 单用户系统（**Single-user systems**）：
 - 多屏并排显示（Multiple side-by-side displays），常用于桌面 VR 工作站或大体积显示（如 CAVE 或“Wall”）。
 - 解决方案：1) 为每个投影仪配备一台带有图形加速卡的独立 PC；2) 一台 PC 配备多张图形加速卡（每个显示器一张）。
- 多用户系统（**Multi-user systems**）：包含客户机-服务器系统（Client-server）、点对点系统（Peer-to-peer）、混合系统

5.2 多管线同步（Multi-pipeline Synchronization）

- 无论管线是在单台 PC 内还是多台 PC 之间，输出图像都必须进行同步，这对于降低系统总延迟并维持一致的帧率至关重要（若不同步，会导致显示画面错位撕裂）。
- 同步的三种层级：
 1. **Software Synchronization**（软件同步）：在应用阶段同步并行管线，使其同时开始处理新帧。缺陷：未考虑各管线实际负载的不对称性
 2. **Frame Buffer Synchronization**（帧缓冲区同步）：同步缓冲区交换（Buffer swapping）。缓冲区交换必须等待显示器的刷新信号
 3. **Video Synchronization**（视频同步）：指定一个主显示器（Master），其余为从显示器（Slave）。通过将从机的内部视频逻辑电路与主机同步，使得从机的垂直和水平刷新扫描线追随主机
- 最佳方案：结合软件同步 + 缓冲区同步 + 视频同步 的综合方案效果最佳

六、PC Clusters（PC 集群）

6.1 大型拼接显示的实施方案

- 当系统（如几十个投影仪拼接的墙体）需要极高分辨率时，单台 PC 无法胜任，原因包括：PCIe 插槽数量不足、总线拥堵导致吞吐量下降
- 解决方案对比：
 1. 大型多管线工作站（如 **PanaWall**）：缺点是成本过高，且单管线仅能支持 1-2 个显示器
 2. 使用 PC 集群（**PC Cluster**）：可以显著降低系统成本；图像拼接（Image tiling）可由软件完成，而非依赖昂贵的专用硬件 PC 需要通过高速 LAN 联网，以便通过控制服务器同步图形输出，局域网的吞吐量是集群规模的主要限制因素

6.2 典型案例与网络架构（WireGL）

- 系统案例：
 - **Princeton Display Wall**：24 台基于 PC 的投影仪拼接，形成 6000x3000 像素的图像，使用 100Mb 以太网连接
 - **Chromium PC Cluster (Stanford)**：由 32 个渲染服务器和 4 个控制服务器组成（带 NVIDIA 显卡），通过 100MB/s 的局域网互联
- **WireGL** 架构机制：

- 这是一个专为集群设计的可扩展驱动程序。WireGL 软件驱动拦截 VR 应用发出的 OpenGL 命令，并在发送至服务器前进行编码
- 命令与顶点数据被打包在一个已知结构的包中发送
- 优化传输 (**Tile Rendering**)：仅向需要的 PC 发送数据 几何数据会根据屏幕空间 (Screen space) 进行排序，使得每个渲染服务器仅获取与其负责输出部分相重叠的几何图形
- 每个渲染服务器都有自己的网络缓冲区，WireGL 跟踪各服务器状态，仅在服务器状态与网络缓冲区之间存在差异时才传输缓冲区数据
- 性能提升：【图表分析表明】随着拼接配置（如 $4 \times 4, 8 \times 4$ ）复杂度增加，基于 WireGL 的网络架构的相对帧率远高于基于广播的网络架构

Turn 2

Assistant

这份笔记是为你考试中“架构 (Architecture)”主题的完美补充。上一份笔记我们已经彻底梳理了“流水线架构 (Rendering Pipeline)”，现在我们将严格根据你最新上传的课件，为你结构化整理出“分布式虚拟环境 (Distributed Virtual Environments)”的完整复习笔记。

格式依然严格遵循三级结构与所有约束条件。

七、 Distributed Virtual Environments (分布式虚拟环境架构)

7.1 DVE 基础概念与分类

- **DVE (Distributed Virtual Environment)** 定义： 一个共享的虚拟环境，允许远程用户通过网络与虚拟对象进行交互，甚至共同完成任务
- **CVE (Collaborative Virtual Environment)** 定义： 支持多个用户在虚拟世界中共同完成任务的特定 DVE
- 典型应用场景： 虚拟博物馆 (Virtual museum)、在线游戏 (Online games)、第二人生 (Second Life)、元宇宙 (Metaverse)

7.2 Multi-Server DVEs (多服务器分布式架构)

- 核心挑战： DVE 强调交互性，用户期望获得如同真实世界般的即时响应 在传统的客户端-服务器 (Client-server) 系统中，随着用户数量增加，单一服务器在处理请求、更新虚拟化身/对象位置以及分发更新消息时会过载，无法即时响应
- 解决方案： 增加更多服务器来分担工作负载
- 三种核心服务器分区架构对比 (重点)：

分区架构类型	核心机制与 workflow	优点	缺点 / 局限性
User Partitioning (用户分区)	将不断加入的用户分配到不同的服务器；当一台满载时，启动新服务器并分配新用户	方法非常简单	不同服务器上的用户通常无法相互交互，相当于各自运行独立的 DVE

分区架构类型	核心机制与 workflow	优点	缺点 / 局限性
Region Partitioning (区域分区)	在 DVE 启动前执行一次；将虚拟环境划分为多个固定区域，每个服务器负责一个区域（空间负载划分）。热门区域划分得较小，冷门区域划分得较大	实现相对固定和明确	如果一个原本冷门的区域突然变得非常热门，对应的服务器依然会过载
Dynamic Partitioning (动态分区)	在系统运行时动态划分区域。如果某区域总负载过高，会重新划分为更小的区域，反之亦然	能够根据实时负载灵活调整资源	重新分区过程需定期执行；受限于实时性要求，负载转移通常只能在相邻服务器之间局部进行

• **Dynamic Partitioning** 的负载转移步骤：

1. 当服务器过载时，首先确定要将额外负载转移给哪些相邻服务器
 2. 过载服务器与选定的相邻服务器通信，获取它们当前的工作负载情况
 3. 根据情况将额外的负载转移给相邻服务器
- **【图示文字化说明：动态分区演变图】** 初始状态下，整个空间被均匀或粗略地划分为多个矩形网格；随着虚拟世界中用户（黑点）数量的大量增加与聚集，人口密集的区域其网格会被动态细分为更小的矩形块，以分散单一服务器的计算压力

7.3 Motion Synchronization（运动同步机制）

- 核心问题： 分布式系统的根本问题是网络延迟 (**Network latency**)，这在强调实时交互的 DVE 中尤为严重
- **【网络延迟参考数据】**： 局域网 (LAN) 约 0.64ms；香港本地互联网约 10ms；美国-香港跨洋连接约 160ms；英国-香港跨洋连接约 203ms
- 分布式系统中的事件分类对比：
 - **Discrete events (离散事件)**： 发生的时间间隔较大（通常远大于网络延迟）。网络延迟对这类事件影响不大。例如：ATM 交互
 - **Continuous events (连续事件)**： 发生的时间间隔非常小（通常远小于网络延迟）。网络延迟对这类事件有显著影响。例如：3D 游戏中的实时交互
- 连续事件的延迟冲突场景（举例）：
 - 用户 A 在时间 t 看到物体在位置 p ，将其移动到 p_A ，并发消息给 B

- 由于网络延迟 0.2s, 用户 B 在时间 $t + 0.1$ 依然以为物体在 p , 将其移动到 p_B 并发消息给 A
- 冲突结果: A 和 B 均收到对方的延迟消息, 引发了状态不一致的矛盾
- 传统同步解决方案及其缺陷:
 - **Causal ordering (因果排序)**: 事件应按发生顺序执行 这对于离散事件可行, 但在多用户 DVE 中, 用户连续生成事件, 因果排序通常不可能实现
 - **Roll-back (回滚机制)**: 例如强制判定 A 获胜, 强行将 B 端的物体从非法位置 p_B 瞬移回正确位置 p_A 。缺点: 容易引发玩家争议和体验割裂
 - **Object locking (对象锁定)**: 阻止其他用户与同一对象交互
- **Dead Reckoning (航位推测法 / 死区预估法)**:
 - 作用: 解决连续事件同步问题的最流行方法
 - 核心原理: 发送端 (生成事件的机器) 和接收端机器均对每个移动物体运行相同的 **Motion predictor (运动预测器)**
 - 工作机制 (输入 / 核心步骤):
 1. 预测器根据最后一次更新消息 (包含物体最新位置、速度/加速度) 外推/预测物体的当前位置
 2. 用户设定一个误差阈值 (**Error threshold value**)
 3. 只要发送端检测到实际位置与预测位置之间的误差小于阈值, 就不发送任何更新消息
 4. 只有当误差大于阈值时, 发送端才会向接收端发送包含最新位置、速度或加速度的更新消息
 - 优点: 避免了定期发送大量位置更新消息, 有效节省了网络带宽

7.4 Motion Prediction (运动预测算法与网络评估)

- 运动预测核心目标: 在已知最后对象位置及其速度/加速度的情况下 (且存在网络延迟), 估算对象的当前位置
- **Polynomial Predictors (多项式预测器算法)**:
 - **FOP (First Order Polynomial / 一阶多项式)**: $p_{new} = p + vt$
 - **SOP (Second Order Polynomial / 二阶多项式)**: $p_{new} = p + vt + at^2/2$

- 公式变量含义:

- p : 来自更新消息的最新物体位置
- p_{new} : 预测的物体位置
- t : 预测的时间长度 (即网络延迟)
- v : 物体移动速度
- a : 物体移动加速度

- 速度与加速度的推导算法:

- 为了估算 v , 需要收集前两帧的位置: $v = [p(t) - p(t - 1)]/dt$
- 为了估算 a , 需要收集前三帧的位置: $a = [p(t) - 2p(t - 1) + p(t - 2)]/dt^2$

- 多项式预测器的优缺点:

- 优点: 非常简单, 计算开销 (Computation overhead) 极小
- 缺点: 不是精确的预测器。它强行假设 v 和 a 已知, 并且假设它们在预测期间保持不变

- **Estimating Network Latency (网络延迟的动态估算):**

- 为了在上述公式中代入 t (延迟), 必须动态估算, 因为网络延迟不是常数, 会随网络流量变化
- 估算机制: Client A 在 T_1 时刻发送消息, Client B 在 T_2 收到并于 T_3 发送回复, Client A 在 T_4 收到回复
- 公式:
 - $round - trip\ delay = (T_4 - T_3) + (T_2 - T_1)$
 - 单向延迟估算: $latency_{A \rightarrow B} \cong latency_{B \rightarrow A} \cong [(T_4 - T_3) + (T_2 - T_1)]/2$
 - 如果处理延迟极小, 可以假设 $T_2 = T_3$ 以简化公式
- 局限性: 必须依赖完整的往返消息 (Round-trip message) 才能确定最新的网络延迟

- **Problems with Prediction (预测机制的固有缺陷):**

- 启动预测失效: 当一个物体刚刚从静止状态开始移动时, 预测必然是错误的, 因为此时之前的 v 和 a 均为零

- 变速影响：大多数物体不会以恒定的 v 和 a 移动。如果物体不断改变其速度和加速度，预测的准确性将受到严重影响

Exported from [Gemini Voyager](https://github.com/Nagi-ovo/gemini-voyager) (https://github.com/Nagi-ovo/gemini-voyager) • 2 conversation turns

Generated on May 6, 2026 at 06:56 PM